Ensembles and random forests

Forest CoverType · one tree you can read, two hundred you cannot, and what averaging can and cannot do

LECTURE 5 | GÉRON CH. 5–6

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Four lectures on one idea: a weighted sum $\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}$, fitted by least squares or by gradient descent, read through a sigmoid for a class — and judged on data it never saw.

> **THE STANDING CONSTRAINT, UNCHANGED**
>
> Split before anything is fitted. All preprocessing inside a `Pipeline` passed to cross-validation. Nothing derived from the test set in the training path. Fixed seeds. Report the fold scores, not only the mean.

Today's model is not a weighted sum at all. It is a sequence of yes/no questions, and the sequence *is* the explanation. Then we fit two hundred of them, and ask what that buys and what it costs.

Today

1. **The task** — a forest-cover map; a decision tree a person can read, and the measurement that shows one tree is not enough
2. **Why an ensemble of trees** is the model practitioners fit on a table — and the one formula that says what averaging can and cannot do
3. **The method** — bagging, random forests and boosting: how to build them, what they cost, how to choose, and which to reach for
4. What the forest changed for the agency, and what it gave up
5. **Design exercise** — the probability of a click on an Instagram ad, in nine steps, with the pitfalls and how to address them

Open Lecture 5 in Colab

The setting

An agency that manages a national forest has to publish a map of **forest cover type** — which species dominates each 30 by 30 metre patch of ground — over an area far too large to survey on foot.

> **THE TASK**
>
> From cartographic measurements alone — elevation, slope, aspect, distances to water, road and fire ignition points, hillshade, soil type — predict which of **seven** tree species covers the patch.

The survey: 581,012 patches, 54 cartographic columns, no missing values. We work on a stratified 60,000-patch subsample, split 48,000 / 12,000 before anything is fitted, and say so.

The map goes into the public record and decides where logging is permitted. A landowner refused a permit is entitled to ask *why*, so the agency wants each prediction to come with a reason a person can read: conditions on the measured columns, **at most eight** of them.

Keep that requirement in mind. It is what the model family of the first half of today satisfies, and what the second half gives up.

Metric before model, and the anchor beside it

Seven classes, no natural ordering, no asymmetric cost stated by the agency. The default is **accuracy** — reported, as always, beside the number that says what *nothing* scores.

```
from sklearn.dummy import DummyClassifier

dummy = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
print(f"{dummy.score(X_test, y_test):.1%}")     # 48.8%
```

Model	Test accuracy	Species it can ever predict
Always “Lodgepole Pine”	48.8%	✗ 1 of 7

✓ Anything below 48.8% is worse than a constant. The distance above it is the only part you actually earned.

What a decision tree is

A binary tree. Each internal node asks one question about one feature; each leaf carries a prediction.

- A question is always of the form $x_k \leq t_k$ — one feature, one threshold
- Prediction: start at the root, answer the question, go left or right, repeat until a leaf
- The leaf predicts the **majority class of the training instances that reached it** — the $\arg\max_k p_k$ of that node's class distribution

No weighted sum anywhere, so no feature scaling: $x_k \leq t_k$ is invariant under any increasing transformation of x_k.

✓ The path from the root to the leaf *is* the justification the agency asked for: one condition per level, in the measured columns. That is what makes a tree readable, and it is free.

How CART chooses a split

A node holds some training instances; write p_k for the fraction in class k. Its **Gini impurity** is

$$G(\mathbf{p}) \;=\; 1 - \sum_{k=1}^{K} p_k^2$$

the probability that two instances drawn from the node, with replacement, differ in class; a pure node scores 0. A split $x_k \le t_k$ sends m_{L} instances left and m_{R} right; CART picks the feature and threshold minimising the size-weighted impurity of the children:

$$J(k, t_k) \;=\; \frac{m_{\mathrm{L}}}{m}\, G_{\mathrm{L}} \;+\; \frac{m_{\mathrm{R}}}{m}\, G_{\mathrm{R}}$$

Greedy, top down, one node at a time, until every leaf is pure or a limit stops it. `criterion="entropy"`, $-\sum_k p_k \log_2 p_k$, ranks splits almost identically; we leave the default.

Fit one, with no constraints at all

```
from sklearn.tree import DecisionTreeClassifier

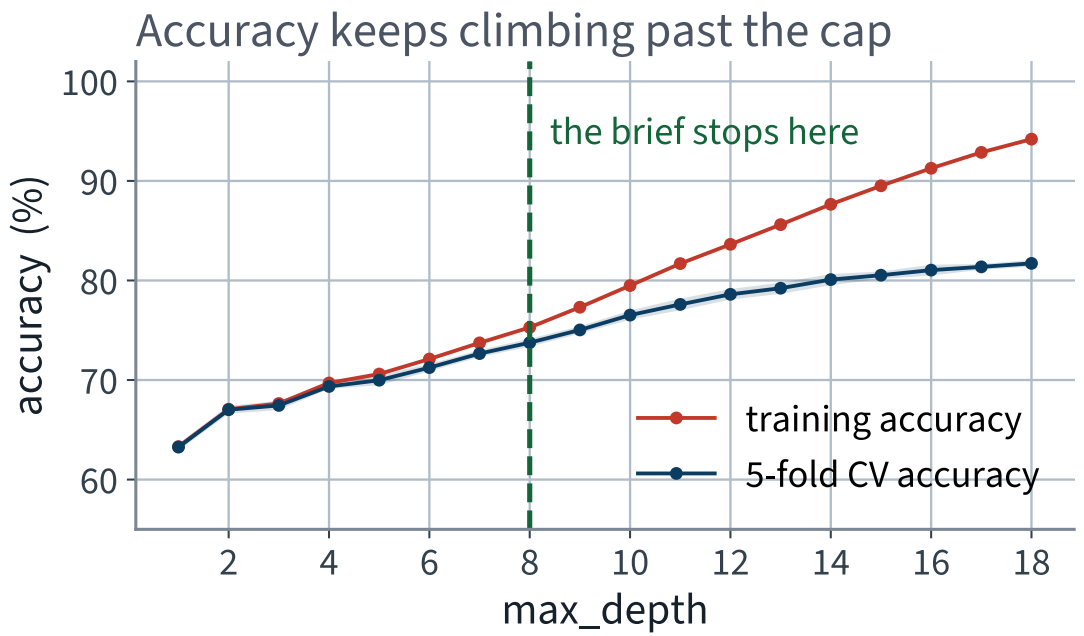
free = DecisionTreeClassifier(random_state=42)      # no depth limit
free.fit(X_train, y_train)

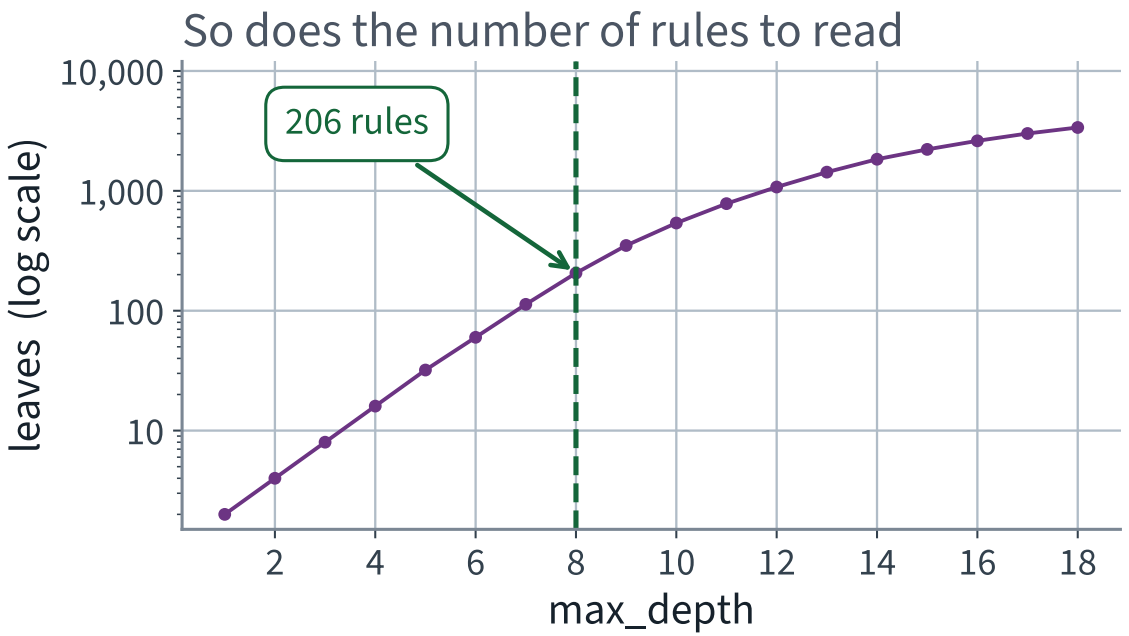
print(f"depth          {free.get_depth()}")
print(f"leaves         {free.get_n_leaves():,}")
print(f"train accuracy {free.score(X_train, y_train):.1%}")
```

```
depth          33
leaves         5,699
train accuracy 100.0%
```

✗ 100.0%: with eight patches per leaf it is perfect on its own data. Cross-validated on the training set it scores 82.1%, and on the test set, looked at once, 82.6% — far above the 48.8% anchor, so trees work here. But it is 33 levels deep, and the agency allows eight conditions.

Accuracy against depth, and the price of reading it





Cross-validated on the training set, depth 1 to 18. The curve is still climbing at depth 18, 8.0 points above depth 8 — and every one of those points is bought with more rules than anyone will read.

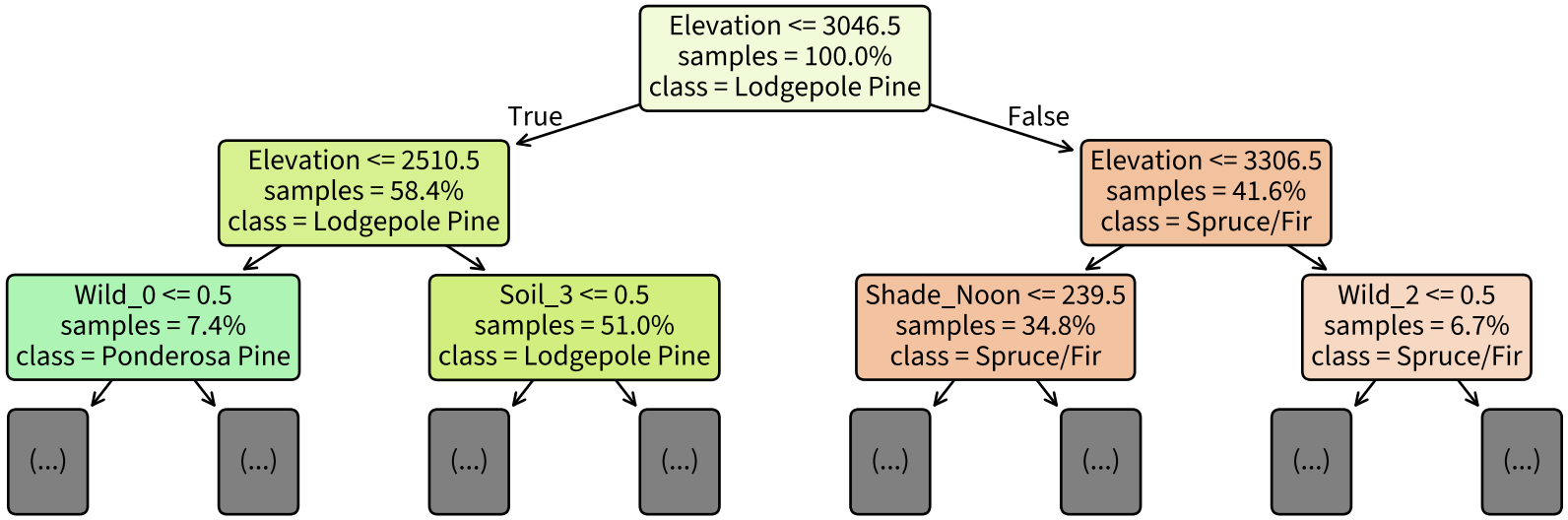
A tree a person can read

```
tree = DecisionTreeClassifier(max_depth=8, min_samples_leaf=20,
                              random_state=42).fit(X_train, y_train)
```

	Unconstrained	Depth 8, leaf 20
Leaves	5,699	**163 ✓**
Mean conditions per prediction	**✗ 17.88**	**7.97 ✓**
Test accuracy	**82.6% ✓**	**✗ 73.0%**

Depth 8 is the brief's eight conditions; a minimum leaf of 20 is so that "91% of the patches in this leaf" is a statement about twenty patches at least, not about one. Readability costs about ten points here, and the agency is told so.

The top three levels of the readable tree



The first three questions the model asks about any patch in Colorado. Two of them are about elevation. A patch's eight answers are its justification — this is the thing to hold on to when we start averaging.

The measurement this lecture turns on

MEASURED IN THE NOTEBOOK, BEFORE ANYTHING IS AVERAGED

Refit the same depth-8 tree, with the same hyperparameters and the same seed, on twenty different 90% subsamples of the same training rows. Test accuracy moves by a standard deviation of **0.25 points** — and **9.1%** of the individual predictions change. Only **71.4%** of test patches get the same answer from all twenty.

A stable metric, an unstable model. This is **variance**: the sensitivity of the fitted function to the particular training sample it happened to get.

✓ The whole of today follows from one question: what does *averaging* do to variance?

Why the members are trees

An **ensemble** aggregates many models, and the book's promise is that enough *sufficiently diverse* weak learners make a strong one. "Sufficiently diverse" is about to get a symbol, a formula and a measured value. First, the members.

- Averaging attacks **variance**, and trees are the high-variance model we have just measured: 9.1% of predictions move when 10% of the rows do
- A tree is cheap to fit, so two hundred of them are affordable
- An unconstrained tree has **low bias** — and averaging does nothing whatever to bias, so the members had better not have any to spare

> That third point is why the ensembles in this lecture use *unconstrained* members. It looks reckless after an opening spent capping the depth at 8, and by the end of the mathematics it will look obvious.

When an ensemble of trees is the tool

The data looks like	Why trees, averaged
A table: numbers and categories mixed, on different scales	a split $x_k \leq t_k$ needs no scaling and no linear shape
Interactions nobody can write down — elevation *and* soil *and* aspect	a path through a tree *is* an interaction, found rather than specified
Tens of thousands of rows or more	enough to grow deep members whose errors differ
Accuracy matters more than a one-line reason	the vote is accurate; the single tree was the readable one

Credit scoring, fraud flags, churn, click prediction, risk scores in medicine, nearly every tabular competition: the same shape of data, and on it a forest or a gradient-boosted ensemble is the first model a practitioner fits.

✗ Not images, text or sequences: a pixel or a token is not a column with a meaning of its own, which is why Parts II–IV exist.

Start with two, then take n

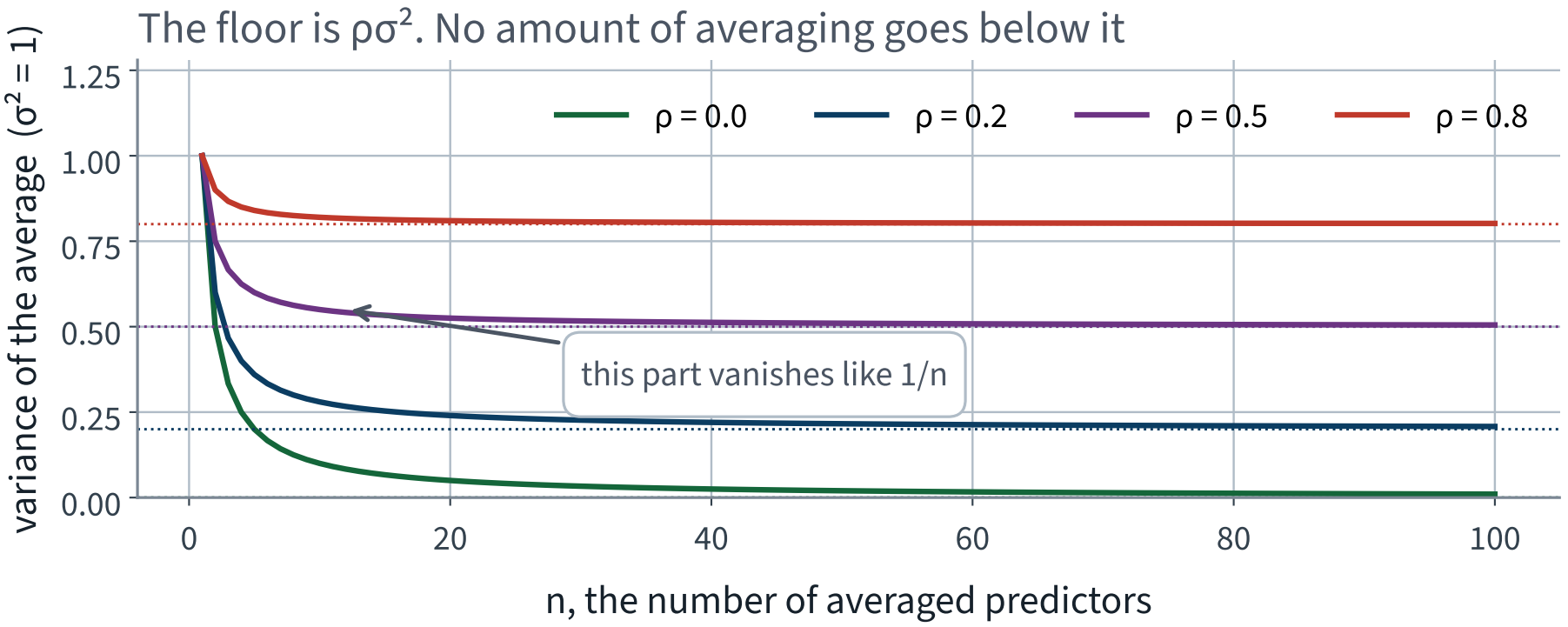
Forget trees for a moment. You have n predictors of the same quantity, each with variance σ^2, and you average them. With $\mathrm{Var}(f_1) = \mathrm{Var}(f_2) = \sigma^2$ and $\mathrm{Corr}(f_1, f_2) = \rho$:

$$\mathrm{Var}\left(\frac{f_1+f_2}{2}\right) = \frac{1}{4}\left[\sigma^2 + \sigma^2 + 2\rho\sigma^2\right] = \frac{\sigma^2(1+\rho)}{2}$$

$\rho = 0$ halves the variance; $\rho = 1$ leaves it at σ^2, and averaging achieves nothing. For n identically distributed predictors with a common pairwise correlation ρ, the same expansion — n variances and $n(n-1)$ covariances $\rho\sigma^2$, over n^2 — gives

$$\mathrm{Var}(\bar{f}) \;=\; \rho\sigma^2 \;+\; \frac{(1-\rho)\,\sigma^2}{n}$$

The same formula, plotted



Four curves, four floors. The floor is set entirely by ρ — which is why every ensemble variant you are about to meet is an attempt to lower it.

What this says before we build anything

1. Averaging many models reduces variance, but only the part they do not share
2. The return on more members collapses quickly. There is little point past the point where $(1-\rho)\sigma^2/n \ll \rho\sigma^2$
3. Therefore the way to improve an ensemble is not more members. It is **less correlated** members
4. And anything that lowers ρ by making individual members worse is a trade, not a free lunch: σ^2 appears in *both* terms, so it can rise while ρ falls

✓ Point 4 is the one that gets forgotten, and it is the one we will measure — and be embarrassed by — later today.

Voting — the simplest ensemble

Combine *different* algorithms fitted to the same data.

```
from sklearn.ensemble import VotingClassifier

vote = VotingClassifier(
    [("tree",   DecisionTreeClassifier(random_state=42)),
     ("logreg", LogisticRegression(max_iter=1000)),
     ("knn",    KNeighborsClassifier(n_neighbors=5))], voting="soft")
```

- `voting="hard"` — majority of the predicted labels
- `voting="soft"` — average of the predicted probabilities; usually better, because a confident member outweighs several hesitant ones

Different algorithms make different kinds of errors, which is the informal way of saying ρ is small. The formula applies unchanged.

Bagging

Bootstrap **agg**regat**ing**. Draw m rows from the training set *with replacement*, fit a tree, and repeat n times.

- Each bootstrap sample has the same size as the training set, but about **63%** of its distinct rows — the rest are duplicates
- Aggregate by majority vote for classification, by the mean for regression
- Each member is individually worse than one tree fitted to the whole set. The ensemble is better — that is the formula, not a paradox

Pasting is the same thing without replacement. It produces less diverse members, so bagging usually wins — but it is a cheap thing to cross-validate rather than assume.

Bagging, in code

```
from sklearn.ensemble import BaggingClassifier

bag = BaggingClassifier(
    DecisionTreeClassifier(random_state=42),   # unconstrained members
    n_estimators=200, bootstrap=True, oob_score=True,
    random_state=42, n_jobs=-1)

bag.fit(X_train, y_train)
print(f"out-of-bag {bag.oob_score_:.1%}")
```

The members are **unconstrained** trees. We want each one to have low bias, because averaging cannot remove bias — and we are about to average the variance away, so there is no reason to restrict their depth.

A validation set you get for nothing

Each member never sees about 37% of the rows — its **out-of-bag** instances. Score each member on its own out-of-bag rows and you have a generalisation estimate with no validation split and no extra fit.

> **WHY 37%**
>
> A given row is missed by one draw with probability $1 - 1/m$, so by all m draws with probability $\left(1 - \frac{1}{m}\right)^m \to e^{-1} \approx 0.368$ as m grows. Hence about 63% distinct rows in, 37% held out.

Cheap, and it does not replace the test set. It replaces the *validation* set — and it estimates one member's procedure, not the ensemble you ship. With `bootstrap=False` there are no out-of-bag rows and `oob_score=True` raises. If it comes out much better than the test score, something in your pipeline saw the out-of-bag rows anyway.

Random forests

Bagging, plus one more source of difference: at **every node of every tree**, only a random subset of the features is considered for the split.

```
from sklearn.ensemble import RandomForestClassifier

rnd = RandomForestClassifier(n_estimators=200, max_features="sqrt",
                             random_state=42, n_jobs=-1).fit(X_train, y_train)
```

`max_features="sqrt"` is the classification default: $\lfloor \sqrt{54} \rfloor = 7$ of the 54 columns, drawn afresh at each node. A tree with 200 nodes therefore draws 200 different subsets.

And what did that do to the accuracy?

200 members, 48,000 training patches	10 members	200 members
Bagging	87.7%	**89.8% ✓**
Random forest	85.9%	88.4%
The legible depth-8 tree	—	73.0%
Always “Lodgepole Pine”	—	48.8%

✓ Both ensembles beat the readable tree by fifteen points and the *unconstrained* one by six. Most of the gain arrives by the first few dozen members, as $(1-\rho)\sigma^2/n$ says it should: the last 190 buy about two points.

ρ, measured — and the uncomfortable row

Measured across 20 disjoint training sets of 20,000 rows, so that the shared part of the variance has something to come from:

Ensemble	ρ	One member's accuracy	Ensemble accuracy
Bagging	✗ 0.072	78.8% ✓	89.8% ✓
Random forest	0.043 ✓	✗ 73.6%	✗ 88.4%

✗ The forest lowers ρ as the formula promised — and the ensemble with the *higher* ρ is the more accurate.

✓ Point 4 of the formula's reading, measured: every mechanism that lowers ρ does it by handicapping the members, σ^2 sits in both terms, and the formula says nothing about bias.

Now the bill

We removed the variance. Ask what happened to the thing the agency asked for.

```
total = sum(t.get_n_leaves() for t in rnd.estimators_)
print(f"{total:,} leaves across the 200 trees")
```

```
1,497,919 leaves across the 200 trees
```

✗ The justification for one prediction is now 200 decision paths and a vote: fifteen points of accuracy over the readable tree, and four orders of magnitude more model to read.

What the forest changed for the agency

	Readable tree	Bagged forest, 200 trees
Test accuracy	73.0%	**89.8% ✓**
Predictions that move when the survey is resampled	**✗ 9.1%**	**the variance cut by nine tenths ✓**
What backs one prediction	**8 conditions ✓**	**✗ 200 paths and a vote**

About seventeen more patches in every hundred mapped right, and a map that stops changing when the survey is resampled. That is the case for the ensemble, and it is why this is the model a practitioner fits on a table.

✓ The cost is the justification. A map does not need one per patch; a refused permit does. Ship the forest for the map, and the readable tree wherever a person has to answer for the decision.

Feature importance, as the library offers it

```
imp = pd.Series(rnd.feature_importances_, index=X_train.columns)
print(imp.sort_values(ascending=False).head(5).round(3))
```

`feature_importances_` sums, over every node that split on a feature, the weighted impurity reduction that split achieved — the impurity decrease ΔI of the opening, accumulated.

It runs, it is fast, and the top of the list is entirely sensible: elevation first, then the distances. Nothing invites suspicion.

✓ Before trusting any ranking, ask what it would look like on a variable you *know* carries no information. If you cannot answer, add one and find out.

Add a control column and refit

```
rng = np.random.default_rng(42)
X_tr2 = X_train.assign(random_decoy=rng.random(len(X_train)))
X_te2 = X_test.assign(random_decoy=rng.random(len(X_test)))

rnd2 = RandomForestClassifier(n_estimators=100, max_features="sqrt",
                              random_state=42, n_jobs=-1).fit(X_tr2, y_train)
```

A column of uniform random numbers. It cannot possibly carry information about which trees grow where. Where does it rank among the 55?

The measurement with the control built in

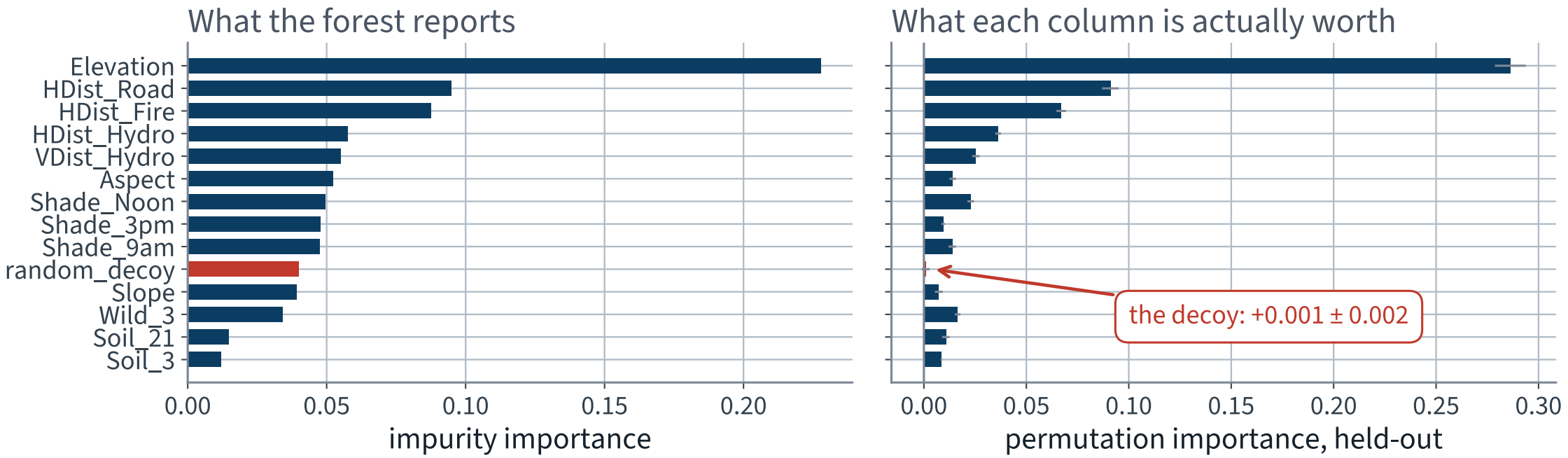
Before the answer, a second measurement to set beside it. Shuffle one column of the **held-out** set, re-score, and see how much accuracy falls. Repeat, and report a spread.

```
from sklearn.inspection import permutation_importance

r = permutation_importance(rnd2, X_te2, y_test, n_repeats=5,
                           random_state=42, n_jobs=-1)
```

> **SCOPE**
>
> `permutation_importance` goes beyond Chapter 6. It is here BEYOND THE SYLLABUS, FOR CONTEXT because the alternative is to ship the biased measurement, and because the idea — break the column and see what breaks — is worth knowing.

The same columns, measured two ways



✓ Same model, same columns, same ordering of the rows. The decoy is tenth on the left and zero on the right.

Boosting — a different idea entirely

Everything so far trains members **in parallel** and averages them. Boosting trains them **in sequence**, each one correcting its predecessor.

> The members are not identically distributed and are certainly not exchangeable, so **today's formula does not apply to boosting**. Bagging reduces variance; boosting reduces *bias*.

✗ Two consequences: boosting cannot be parallelised across members, and it is much easier to overfit with. Both follow from the sequence.

AdaBoost

1. Fit a weak learner — typically a tree of depth 1, a *decision stump*
2. **Increase the weight** of the instances it got wrong
3. Fit the next learner on the reweighted data; repeat
4. Predict by a weighted vote, each learner weighted by its own accuracy

```
from sklearn.ensemble import AdaBoostClassifier

ada = AdaBoostClassifier(DecisionTreeClassifier(max_depth=1),
                         n_estimators=200, learning_rate=0.5,
                         random_state=42)
```

If it overfits, reduce `n_estimators` or `learning_rate`, or regularise the base estimator.

Gradient boosting

Same sequence, different correction. Instead of reweighting the instances, fit each new learner to the **residual errors** of the ensemble so far.

- `learning_rate` scales each member's contribution. A low rate needs more members and generalises better — this is *shrinkage*
- Too few members underfits; too many overfits. Use early stopping with `n_iter_no_change`
- The number of members is therefore *not* a "more is better" parameter, which is exactly the opposite of bagging

> **WHEN IT BITES**
>
> Adding trees to a bagged ensemble can only help, because $(1 - \rho)\sigma^2/n$ falls monotonically. Adding trees to a boosted ensemble eventually hurts. Carrying the first intuition into the second is one of the commonest mistakes with these models.

What practitioners actually reach for

`HistGradientBoostingClassifier` bins each feature into at most 255 buckets before growing anything, so a split search costs $O(\text{bins})$ rather than $O(m)$.

```
from sklearn.ensemble import HistGradientBoostingClassifier

hgb = HistGradientBoostingClassifier(max_iter=100, learning_rate=0.05,
                                     early_stopping=False,
                                     random_state=42).fit(X_train, y_train)
```

- Dramatically faster on a dataset this size, and it is the same algorithm LightGBM and XGBoost implement
- Handles missing values and categorical columns natively — no imputer, no encoder, up to `max_bins` (255) categories per column
- On tabular data it is the usual first choice, and on most benchmarks it is the strongest single model

✗ On *this* dataset, at a budget that fits a lecture, it lands several points below the bagged forest. The notebook measures both, and measures what happens when you raise the learning rate. Usual is not always.

A forest in practice: the recipe that usually works

```
rf  = RandomForestClassifier(n_estimators=300, max_features="sqrt",
                             oob_score=True, n_jobs=-1, random_state=42)
hgb = HistGradientBoostingClassifier(learning_rate=0.05, max_iter=500,
                                     early_stopping=True, random_state=42)
```

- No scaling; `HistGradientBoosting` takes missing values and categories as they are
- A forest: more members never hurt, so set n by the budget and read the out-of-bag score as your validation
- Boosting: tune `learning_rate`, let early stopping pick the number of rounds
- Imbalance: `class_weight="balanced"`, and per-class recall beside the accuracy
- Then the honest part, unchanged since Lecture 2: cross-validate the choice, read the test set once

Which to reach for

Situation	Reach for
A high-variance base learner and time to spare	bagging or a random forest
Many weakly informative columns	a random forest — feature subsampling earns its keep
You want the strongest tabular model	histogram gradient boosting — usually, and measure it against a forest anyway
Several strong, different models and time to validate a blender	stacking — `StackingClassifier`, whose blender sees only cross-validated predictions; built by hand it memorises
You must justify individual predictions	**✗ none of them — go back to one tree**

The last row is the one this lecture exists to teach, and it is the row people skip.

Did the averaging fix what we diagnosed?

The diagnosis was *instability*, not inaccuracy. So test the thing that was diagnosed, not the thing that is easy to measure.

Random forest, 20 disjoint training sets	Measured
Variance of one member's correctness	**✗ 0.16734**
Variance of the average of 20	**0.01508 ✓**
Share of the variance the averaging removed	**91.0% ✓**
The floor $\rho\sigma^2$ it cannot reach below	0.00721

✓ Nine tenths of a single member's variance, gone — and we knew before starting both that it would stop and roughly where. The notebook repeats the twenty-subsample disagreement measurement on a forest; predict the answer before you run it.

The notebook for this lecture

Open Lecture 5 in Colab

- **Runs on free CPU.** The download is about 11 MB
- Everything on today's slides, on the same rows with the same seed: the unconstrained and the readable tree, the depth sweep, the twenty refitted trees, the 200-member pools, the twenty disjoint training sets and ρ measured on them, the decoy column, both importance measurements and the boosted models
- It checks the formula twice — once on synthetic predictors with a ρ you choose, and once against the two measured ensembles

> **WHAT TO CHANGE BEFORE THE NEXT LECTURE**
>
> Set `max_features=None` on the random forest, which turns it back into bagging. Watch ρ rise, each member get better, and the ensemble accuracy rise with it — the trade of the whole lecture, in one keyword.

Design exercise · in the manner of an ML system-design interview

The task: the probability that a user clicks an Instagram ad

> **THE BRIEF**
>
> Every time a feed is loaded, an auction chooses which ad fills the slot. For each candidate (user, ad, context) the system must estimate, in a few milliseconds, the **probability of a click**. The auction ranks candidates by *bid* × *probability*, so the number has to be a probability, not a score: a model that ranks well but misprices every bid loses money on every impression.

- **Given:** the log of past impressions — who saw which ad, where, when, and whether a click followed — plus what is known about the user, the ad and the moment
- **Scale:** billions of impressions a day; clicks are rare, of the order of one impression in a hundred
- **Consumers of the output:** the auction, budget pacing, and the advertiser's reports

✓ Nine steps follow, each with the pitfall that catches a first attempt and what to do about it. The model at the centre is today's: an ensemble of trees on a table.

Step 1 — Frame it honestly: what is predicted, and how it is scored

- One row per impression; the label is *click within the attribution window*; the output is $p(\text{click} \mid \text{user}, \text{ad}, \text{context})$
- The metric is a **proper scoring rule** — log loss — read beside its anchor, the log loss of always predicting the base rate; AUC beside it for the ranking, never instead
- The baseline before any model: the ad's own historical click rate, shrunk towards its campaign's when it has few impressions

> **PITFALL**
>
> With one click in a hundred, "no click" is right ninety-nine times out of a hundred, so accuracy says nothing, and a model can have a splendid AUC while every probability it emits is twice too large. Score with log loss and a calibration plot, against the base-rate anchor, or the first number you report will be meaningless.

Step 2 — The label is not as clean as it looks

Pitfall	What happens	What to do
Delayed feedback	a click can arrive minutes or hours after the impression; a row logged too early is a negative that is not one	label only impressions older than the attribution window, or model the delay explicitly
Invalid traffic	bots and accidental taps are clicks in the log and not in the world	filter with the platform's invalid-traffic rules before anything is fitted
Unseen impressions	an ad logged below the fold was never a candidate for a click	keep viewable impressions only, and say so in the definition

✓ **Write the label's definition down before the first fit. Every later number is a number about *that* label.**

Step 3 — Split before you look, and split by *time*

- Train on the days up to a cut, validate on the next day, test on the day after that — the order the system will actually face
- Choose everything — features, rounds, calibration — on the validation day; read the test day once

PITFALL

A random split puts the same user, the same campaign and the same hour on both sides. The model learns a campaign's clicks of one evening and is scored on the next morning's impressions of it: the offline score climbs, and the gain vanishes the day the model goes live. Split by time, and keep the gap between training and test at least as long as the attribution window, or the delayed labels leak across it too.

Step 4 — Features: let the trees find the interactions

- **User:** coarse demographics, past behaviour as counts and rates — impressions, clicks, time since the last click
- **Ad:** campaign, advertiser, creative type, its own click history; **context:** position in the feed, time of day, device, connection
- No scaling, no manual crosses: a path through a boosted tree *is* “this advertiser × this device × this hour”, found rather than written down — today’s argument for the model family

PITFALL — LEAKAGE THROUGH A FEATURE

“The ad’s click rate” computed over the whole log contains the row’s own click and every click after it. Compute every history feature *as of* the impression’s moment, from the past only, with the same code that will compute it when the ad is served — a feature the serving system cannot produce in time is not a feature.

Step 5 — One click in a hundred: sample, then put the probabilities back

- Billions of rows, almost all negatives: keep every click and a random fraction of the non-clicks, and training fits in the budget
- A model fitted on that sample sees too many clicks, so its probabilities are too high *by a known amount*: shift its log-odds back by the log of the sampling rate, or re-fit the calibration on an unsampled slice

> **PITFALL**
>
> `class_weight="balanced"` fixes the ranking and wrecks the probabilities: it tells the model that clicks are as common as non-clicks, and the auction then overbids on everything. Reweighting is a tool for a classifier that outputs a label; this system outputs a price.

Step 6 — The model: a boosted ensemble of trees, then early stopping

- Tabular rows, mixed types, interactions nobody can write down, tens of millions of rows even after sampling: the case for an ensemble of trees from an hour ago, exactly
- Gradient boosting over bagging here: shallow members are cheap to evaluate, and the model is evaluated billions of times a day; a forest of two hundred deep trees is two hundred paths per impression
- Tune the learning rate on the validation day and let early stopping choose the number of rounds; the ablation table is the record of what each feature family bought

> **PITFALL**
>
> More rounds always lower the training loss and, past a point, raise the next day's: the opposite of bagging, where more members never hurt. Stop on the validation day's log loss, not on the training curve.

What the large platforms do next — the trees as feature generators for a linear or neural layer — starts from this model and belongs to Part II.

Step 7 — Calibration: the auction needs the number to mean what it says

- Bucket the validation day by predicted probability and compare each bucket’s prediction with its observed click rate: the reliability diagram, now a business requirement
- If it bends, fit a calibration layer — isotonic or Platt — on a slice the model never trained on, and check it per segment: new ads, new users, each device

> **PITFALL — THE COLD START**
>
> A new ad has no history, so every history feature is empty and the tree sends it down the branch for “unknown”. Back it off to its campaign’s and advertiser’s rates, and give new ads a small, deliberate share of impressions so that their own rate can be learned.

Step 8 — The model chooses its own training data

- The log holds only the impressions the *previous* model chose to show. An ad it never shows gets no clicks, so the next model never learns to show it: the loop closes on itself
- Keep a small randomised slice of impressions outside the model's choice, and train and evaluate on it too
- Retrain daily; monitor the input distributions and the calibration against the base rate, which drifts with seasons, campaigns and the news

> **PITFALL**
>
> An offline gain is a hypothesis. A model that lowers the validation day's log loss changes which ads are shown, which changes the users, the advertisers and the log. The only test that counts is an A/B test on live traffic, read on revenue and on what users do afterwards, with the budgets of both arms held equal.

Step 9 — Ship it: the pipeline, and the pitfalls in one table

Log → viewable impressions, invalid traffic out, labels after the window → time split → history features as of each impression → sampled negatives → boosted trees with early stopping → probabilities corrected and calibrated → log loss and reliability against the anchor → A/B test → retrain and monitor.

Pitfall	Address it by
Accuracy, or AUC alone	log loss against the base-rate anchor, and a reliability diagram
Clicks that arrive late	labelling after the attribution window
A random split	splitting by time, with a gap
History features that see the future	as-of computation, in the serving pipeline's code
Sampling or reweighting the negatives	correcting the log-odds; never `class_weight` for a price
Too many boosting rounds	early stopping on the validation day
New ads, miscalibrated segments	back-off to campaign rates; calibration per segment
The model trains on its own choices	a randomised slice; A/B tests on live traffic

What we did

1. Grew a decision tree on 48,000 patches and capped it at depth 8 so a prediction is eight readable conditions: 73.0% against 82.6% unconstrained
2. Measured why one tree is not enough: refit on 90% of the rows, 9.1% of its predictions change
3. $\mathrm{Var}(\bar{f}) = \rho\sigma^2 + (1-\rho)\sigma^2/n$: averaging removes the variance the members do not share, so the way to a better ensemble is *less correlated* members, not more of them
4. Built bagging and random forests — rows, then columns — and measured 88–90% against the tree's 73.0%, most of it by the first few dozen members
5. Why this is the model for a table: no scaling, interactions found rather than specified, strong defaults; histogram gradient boosting when you want the strongest tabular model
6. The bill: 1,497,919 leaves and no per-prediction justification — the forest for the map, the readable tree where a person answers for the decision
7. Impurity importance ranked pure noise 10th of 55; permutation importance on held-out data put it at zero
8. Designed the click-probability model behind an ad auction in nine steps — a proper score against its anchor, a label defined after the attribution window, a split by time, as-of features, sampled negatives with the log-odds put back, boosted trees stopped early, calibration, a randomised slice and an A/B test — each step named for the pitfall it avoids

In the book

Topic	Chapter
Voting classifiers, hard and soft	6
Bagging, pasting, out-of-bag evaluation	6
Random forests, feature importance	6
AdaBoost, gradient boosting, histogram-based boosting	6
Stacking and the blender	6
Decision trees, Gini impurity, the CART algorithm	5

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 5

five, in the style of the written paper

Exercises — Lecture 5

1 For n predictors each of variance σ^2 and pairwise correlation ρ, the variance of their average is $\rho\sigma^2 + \frac{1-\rho}{n}\sigma^2$. State the limit as $n \to \infty$, and what it means for ensembles. [5]

2 Bagging measures $\rho = 0.072$ with one member at 78.8% accuracy; the random forest measures $\rho = 0.043$ with one member at 73.6%. Yet bagging's ensemble is the more accurate. Explain, using the formula above. [5]

3 You set `bootstrap=False` and `oob_score=True`. State what happens and why. [3]

Solutions on Lecture 6's deck.

Exercises — Lecture 5 (continued)

4 Impurity importance ranks a column of random numbers tenth of 55; permutation importance on held-out data puts it at zero. Describe how permutation importance is computed, and why a random column cannot score well on it. [5]

5 A forest improves accuracy over one tree by far less than the variance reduction would suggest. Is this a contradiction? Answer with a reason. [4]

Solutions on Lecture 6's deck.

THE FIVE SET AT THE END OF LECTURE 4

Solutions Lecture 4

not part of this lecture

Solutions — Lecture 4

1. Gradient descent on the same data, with the features unscaled, takes many more steps to converge than on…

✓ Unscaled features make the contours elongated, so the gradient points across the valley rather than along it.

- The Hessian is $X^\top X$, so the curvature along a feature goes as the *square* of its scale — multiply a column by ten and its curvature rises a hundredfold, which is how one column comes to dominate
- The learning rate must then suit the steepest direction, which makes it far too small for the shallow one

2. On the Titanic manifest, your logistic regression on one-hot encoded columns produces coefficients that look…

✓ Five come from the encoder, one from you. The repair is both: drop one level per categorical, and delete the engineered column.

- Each one-hot block sums to 1 in every row, and so does the intercept column — five categoricals, five exact dependencies
- The sixth is `FamilySize = SibSp + Parch + 1`, an identity rather than a correlation. Dropping a level alone still leaves the rank one short of the column count, because the dependency you wrote yourself survives it

Solutions — Lecture 4 (2)

3. A model reaches 0.80 accuracy and is badly calibrated. Explain what that means, and name a decision it would…

✓ Its predicted probabilities do not match observed frequencies; any decision using a cost-weighted threshold is then wrong.

- Accuracy reads only the arg-max, so it is blind to the probability that produced it
- Choosing a threshold from expected cost requires the probability to mean what it says

4. In a logistic regression with first class as the reference level, the coefficient on Pclass_3 is -1.169….

✓ It multiplies the *odds* of survival by $e^{-1.169} = 0.31$, relative to first class, holding the other features fixed.

- The linear part predicts the log-odds, so adding 1 to an indicator adds its coefficient to the log-odds and multiplies the odds by its exponential
- The probability moves by an amount that depends on where it started: odds of 1 times 0.31 give $p = 0.24$, while $p = 0.9$ becomes 0.74

Solutions — Lecture 4 (3)

5. State the update rule for batch gradient descent, and say what changes in it for stochastic gradient descent…

✓ $\theta \leftarrow \theta - \eta \nabla J$. Only the set the gradient is computed over changes; the rule does not.

- SGD estimates the same gradient from one instance rather than all of them
- The estimate is unbiased — averaged over a uniformly drawn row it is exactly the batch gradient — but noisy, and it costs $O(n)$ per step instead of $O(mn)$

LECTURE 6 · GÉRON CH. 7–8

Dimensionality reduction and unsupervised learning

What unsupervised learning is, and four tasks from practice, each matched to the one tool that fits
PCA, k-means and DBSCAN on a worked example, and what to look out for
Anomalies, where to spend forty labels, and a design exercise

Run the Lecture 5 notebook first